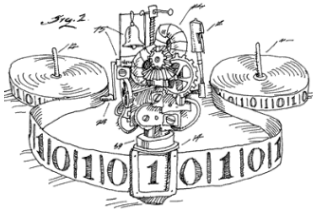
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 6. Lenguajes Regulares y Expresiones Regulares

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 6

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lenguajes Regulares y Gramáticas Tipo 3

Expresiones Regulares

Equivalencia y Conversiones

Minimización de Autómatas

Aplicaciones en Python

Aplicaciones en SQL

Conclusión

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lenguajes Regulares y Gramáticas Tipo 3: El Crecimiento de las Cadenas

Las Gramáticas Lineales

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición de Gramática Lineal a la Derecha

Toda regla de producción tiene la forma:

$$A \mapsto aB \quad \text{o} \quad A \mapsto a \quad (\text{donde } A, B \in V_n,\ a \in V_t)$$

La Derivación como el avance del proceso

En el análisis secuencial, cada paso de la gramática lineal a la derecha genera un símbolo terminal por el extremo izquierdo, mientras que el único no-terminal "viaja" hacia el final:

$$S \mapsto \text{cargar} \cdot L \mapsto \text{cargar} \cdot \text{iniciar} \cdot P \mapsto \text{cargar} \cdot \text{iniciar} \cdot \text{fin}$$

El no-terminal activo representa el **puntero del estado de ejecución** actual.

De Autómatas (DFA) a Gramáticas Lineales

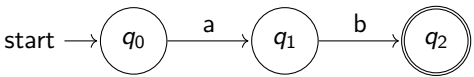
UNIVERSIDAD DE LOS ANDES VENEZUELA

La Equivalencia Formal

Todo Autómata Finito Determinista puede convertirse directamente en una gramática regular equivalente siguiendo estas reglas:

- Cada estado q_i del autómata se convierte en un símbolo no-terminal A_i.
- Cada transición $\delta(q_i, a) = q_j$ se convierte en la producción: $A_i \mapsto aA_j$.
- Si q_i es un estado final de aceptación, agregamos la producción vacía: $A_i \mapsto \varepsilon$.

El autómata



$q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$

Lenguaje es: ab

Ejemplo: Gramática de la Estación de Pintura

UNIVERSIDAD DE LOS ANDES VENEZUELA

Grafo a Producciones

A partir de las transiciones del recurso de pintura (Sesión 5), definimos las reglas de producción en BNF lineal:

$$
\begin{aligned}
\langle \text{OCIOSO} \rangle &::= \text{cargar } \langle \text{LISTO} \rangle \\
\langle \text{LISTO} \rangle &::= \text{iniciar } \langle \text{PINTANDO} \rangle \\
\langle \text{PINTANDO} \rangle &::= \text{fin } \langle \text{COMPLETADO} \rangle \mid \text{detectar_falla } \langle \text{FALLA_TEMPORAL} \rangle \\
\langle \text{FALLA_TEMPORAL} \rangle &::= \text{reparar } \langle \text{LISTO} \rangle \mid \text{sobrecalentar } \langle \text{FALLA_CRITICA} \rangle \\
\langle \text{COMPLETADO} \rangle &::= \varepsilon \mid \text{despachar } \langle \text{OCIOSO} \rangle \\
\langle \text{FALLA_CRITICA} \rangle &::= \varepsilon
\end{aligned}
$$

Expresiones Regulares (Regex): El metalenguaje de patrones

Traducción: De la Matemática al Código

Equivalencia conceptual para diseño de Workflows

Concepto Formal	Símbolo	En Programación / APIs	Sentido en el Workflow
Alfabeto	Σ	Catálogo de eventos / métodos	Menú de tareas permitidas en la planta/oficina.
Cadena	w	Traza de auditoría / Archivo Log	Película temporal de eventos registrados en orden.
Concatenación	$\cdot$	Concatenación de strings ($A + B$)	Secuencia en el tiempo: *"hacer A y luego hacer B"*.
Cadena Vacía	ϵ o λ	Transición automática / Bypass	Paso lógico de control instantáneo (no consume tiempo).
Clausura de Kleene	$*$	Bucle infinito o condicional (*while*)	Ciclo de retrabajo: *"corregir cero o más veces"*.
Lenguaje	L	Reglas sintácticas de validación	El conjunto de trazas que completan el proceso con éxito.

Ejemplo de analogía

La Regex para comprar en línea: solicitar $\cdot$ (agregar_item)* $\cdot$ pagar
Describe que el cliente solicita el carro, agrega ítems cuantas veces quiera (incluso cero), y luego paga de forma secuencial obligatoria.

Definición de Expresiones Regulares

UNIVERSIDAD DE LOS ANDES VENEZUELA

Concepto

Una Expresión Regular (Regex) es una forma compacta y algebraica de describir un Lenguaje Regular. Utiliza operadores matemáticos básicos para definir patrones de cadenas aceptables:

- **Alternancia (o):** Representado por $|$ o $+$.
- **Concatenación:** Símbolos colocados uno tras otro.
- **Cierre de Kleene (*):** Cero o más repeticiones de un patrón.

Mapeo algebraico de la pintura

La secuencia básica exitosa de la estación de pintura se puede resumir como:

$$\text{cargar} \cdot \text{iniciar} \cdot (\text{detectar_falla} \cdot \text{reparar} \cdot \text{iniciar})^* \cdot \text{fin}$$

UNIVERSIDAD DE LOS ANDES VENEZUELA

Equivalencia y Conversiones: De la Descripción al Autómata

Equivalencia entre Expresiones Regulares y Autómatas

UNIVERSIDAD DE LOS ANDES VENEZUELA

Teorema de Kleene

Un lenguaje es regular si y solo si es aceptado por algún autómata finito (DFA o NFA) y si y solo si es descrito por alguna expresión regular.

De Regex a Autómata (Compilación)

Útil cuando diseñamos un workflow como una expresión regular (ej: un pedido requiere solicitar, evaluar, y luego enviar) y queremos generar el motor lógico (DFA) que lo ejecute.

De Autómata a Regex (Auditoría)

Útil cuando tenemos la lógica del recurso físico o de negocio (DFA) y queremos extraer su comportamiento global en una fórmula Regex simple para auditar bitácoras.

Algoritmos de Conversión

UNIVERSIDAD DE LOS ANDES VENEZUELA

De Expresión Regular a Autómata (Algoritmo de Thompson)

- Convierte inductivamente sub-expresiones de la Regex en fragmentos de autómatas con transiciones vacías (ϵ-NFA).
- Permite çompilar.[el] patrón lineal del workflow a una red estructurada de transiciones lógicas.

Determinización (Método de los Subconjuntos)

Convierte el ϵ-NFA resultante de Thompson en un DFA equivalente en el que no existe ambigüedad en el camino de transiciones ante un evento.

UNIVERSIDAD DE LOS ANDES VENEZUELA

Minimización de Autómatas: Reduciendo la Redundancia

¿Por qué minimizar un Autómata?

UNIVERSIDAD DE LOS ANDES VENEZUELA

El problema de la redundancia en Workflows

Al modelar flujos de trabajo de negocio o manufactura, es común crear estados redundantes.

Ejemplo: Tener dos estados distintos en una oficina, `Evaluar_Nivel_1` y `Evaluar_Nivel_2`, que responden exactamente igual a todos los eventos futuros (aprobación, rechazo, cancelación).

La Minimización

Consiste en agrupar los estados equivalentes en un único "estado de clase de equivalencia", obteniendo el autómata con el **número mínimo de estados** posible que reconoce el mismo lenguaje. Esto ahorra memoria en el motor y simplifica la supervisión del proceso.

Estados Equivalentes e Indistinguibles

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición de Equivalencia

Dos estados $p, q \in Q$ son equivalentes (indistinguibles) si para cualquier cadena $w \in \Sigma^*$, al procesarla desde p o desde q, el autómata converge al mismo resultado (ambos aceptan o ambos rechazan):

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

La Relación de Equivalencia

El espacio de estados Q se particiona en clases de equivalencia. Todos los estados dentro de una misma clase se fusionan en un único estado en el autómata mínimo resultante.

El Algoritmo de Partición (Hopcroft / Myhill-Nerode)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Pasos del Algoritmo

1. **Partición Inicial (P_0):** Dividir los estados en dos grupos: Estados de Aceptación (F) y Estados de no aceptación ($Q \setminus F$).
$$P_0 = \{F,\ Q \setminus F\}$$
2. **Refinamiento (P_{k+1}):** Para cada grupo en la partición, verificar si sus miembros transitan a los mismos grupos ante cada símbolo de Σ. Si un estado transita a un grupo diferente, se divide el grupo.
3. **Criterio de Parada:** Detenerse cuando la partición no sufra más divisiones ($P_{k+1} = P_k$).

Tarea: Revisar del texto (Jurado Málaga, 2008) las secciones 4.1 a 4.4

UNIVERSIDAD DE LOS ANDES VENEZUELA

Aplicaciones Prácticas: Expresiones Regulares en Python

Regex en Python: El módulo `re`

UNIVERSIDAD DE LOS ANDES VENEZUELA

Uso en auditorías de logs de procesos

En Python, las expresiones regulares nos permiten validar y extraer información de trazas de eventos rápidamente:

```
import re

# Traza de auditoria de un recurso
log_line = "2026-08-03 11:15:30 [Estacion_Pintura] EVENTO: detectar_falla en mod

# Patron para identificar fallas en cualquier recurso
patron_falla = r"\[(?P<recurso>\w+)\] EVENTO: (?P<evento>\w+_falla|\w+_error)"

match = re.search(patron_falla, log_line)
if match:
    print(f"Falla detectada en: {match.group('recurso')}")
    print(f"Tipo de Evento: {match.group('evento')}")
```

Regex en Python: Validación de Sintaxis

UNIVERSIDAD DE LOS ANDES VENEZUELA

Validación de tokens de productos

Validar si el código de un producto sigue la nomenclatura estructurada de la ontología:

```
import re

# Codigo de producto: TK-seguido de 3 digitos, guion, y una categoria (A, B o C)
codigos = ["TK-901-A", "TK-002-B", "INVALID-123", "TK-99-C"]

patron_codigo = r"^TK-\d{3}-[A-C]$"

for c in codigos:
    if re.match(patron_codigo, c):
        print(f"Codigo valido: {c}")
    else:
        print(f"Codigo INVALIDO: {c}")
```

UNIVERSIDAD DE LOS ANDES VENEZUELA

Aplicaciones Prácticas: Filtrado y Auditoría con Regex en SQL

Regex en SQL: Filtrado de Tablas de Bitácoras

UNIVERSIDAD DE LOS ANDES VENEZUELA

Consultas Relacionales (LIKE vs. REGEXP)

Al auditar bases de datos relacionales de sistemas MES/ERP, los operadores clásicos como `LIKE` se quedan cortos. Usamos expresiones regulares nativas (`REGEXP` o `REGEXP_LIKE`):

```
-- Buscar todas las trazas de eventos que terminen en falla o error
SELECT id_evento, timestamp, descripcion
FROM logs_procesos
WHERE descripcion REGEXP '(falla|error)$';
```

Ejemplo de extracción jerárquica

Extraer logs asociados solo a sub-procesos del recurso R1 (ej: códigos con formato R1.1, R1.2):

```
SELECT * FROM eventos_produccion
WHERE id_recurso REGEXP '^R1\.[0-9]+';
```

SQL: Limpieza de Datos Ontológicos

UNIVERSIDAD DE LOS ANDES VENEZUELA

Validación en base de datos

Garantizar la consistencia de datos de inventario auditando los códigos de barra antes de iniciar el proceso de manufactura:

```
-- Identificar registros corruptos en la tabla de productos
SELECT id_producto, nombre
FROM inventario_productos
WHERE NOT REGEXP_LIKE(codigo_barras, '^PROD-[0-9]{4}-[A-Z]{2}$');
```

Conclusión de la Sesión 6

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lo que aprendimos hoy

- **Crecimiento de Cadenas:** Las gramáticas lineales a la derecha mapean de forma natural la traza secuencial de eventos del recurso.
- **Expresiones Regulares (Regex):** Son el lenguaje formal algebraico para definir y validar patrones.
- **Aplicación en Python:** El uso de *re* permite procesar logs y validar estructuras de forma eficiente en los programas de simulación.
- **Aplicación en SQL:** Las expresiones regulares facilitan consultas complejas y la limpieza de datos de auditoría de planta y oficina.

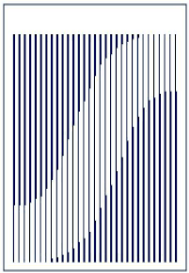
Próxima sesión

Consolidaremos el **Hito 1 (Semana 6)**: cargaremos la especificación del autómata desde YAML y realizaremos la validación de secuencias completas en nuestro motor de consola.

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve

Jurado Málaga, E. (2008). *Teoría de autómatas y lenguajes formales*. Universidad de Extremadura. Servicio de Publicaciones.